SWAGGER-FIRST EXECUTION GUIDE

Notification API Smoke Test Guide

Business workflow validation with expected API output, RabbitMQ data flow, queue observations, retries, and dead-letter behavior.

API: Notification Service API v2

Swagger: `http://localhost:8080/swagger`

RabbitMQ Management: `http://localhost:15672`

Prepared: 07 August 2026

Expected duration: 20–30 minutes including broker tests

```
Swagger POST order → API → SQL job/items → RabbitMQ command queue
RabbitMQ → Worker → SQL notification records → manual ACK
Admin status change → API → SQL → admin + visitor notifications
Visitor cancellation → API → SQL → one admin notification
Repeated processing failure → retry 1 → retry 2 → DLX → DLQ
```

1. Why this smoke test exists

A smoke test answers a focused question: "Is the deployed system healthy enough for deeper testing?" This guide checks the smallest set of high-value behaviors that cross HTTP routing, request validation, SQL persistence, RabbitMQ, the Worker Service, and notification CRUD.

> **Why Swagger:** Swagger uses the live OpenAPI contract generated by the running API. It exposes valid routes, request schemas, enum fields, response codes, and response headers without requiring a frontend application or separate API client.

Required business coverage

Behavior	Expected outcome	Broker involvement
Order placement	The cart becomes an order; a job is accepted; the worker creates admin and visitor notification records.	Yes—publish, consume, process, ACK.
Admin changes order details	Order, payment, fulfillment, and delivery statuses persist; new notifications are created for admin and visitor.	No—synchronous database path.
Visitor cancellation	The owning visitor cancels; other tracking values remain; one admin notification is created; repeat is idempotent.	No—synchronous database path.
Get notifications	List/filter and get-by-ID return persisted notification data.	No.
Modify notification	Full replacement updates details/read state.	No.
Delete notifications	Individual and visitor-wide deletion work; orders and unrelated records remain.	No.
RabbitMQ reliability	Queued work survives a stopped worker; failures retry twice and then reach the DLQ.	Yes—queue, ACK, retry, DLX/DLQ.

> **Event semantics:** An order change does not overwrite the original "order received" notification. It creates a new status-change or cancellation notification. Actual mutation of a notification record is tested separately through `PUT /api/v2/notifications/{id}`.

2. Preconditions and startup

- □ Docker Desktop is running with Linux containers.
- □ `notification-api/.env` contains valid SQL Server and RabbitMQ credentials.
- □ Ports 8080 and 15672 are available.
- □ The API, worker, SQL Server, and RabbitMQ containers are healthy.

```
docker compose --env-file notification-api/.env `
  -f notification-api/compose.week3.yml up --build --detach

docker compose --env-file notification-api/.env `
  -f notification-api/compose.week3.yml ps
```

Open Swagger at `http://localhost:8080/swagger`. In the document selector, choose **Notification Service API v2**. Open an operation, click **Try it out**, enter the values shown in this guide, and click **Execute**.

3. Test data and values to record

> **Use a unique visitor email:** Replace the timestamp below before every run. Reusing an email can make notification counts differ from the expected output.

Purpose	Sample value
Smoke visitor	`swagger.smoke.202608071530@example.test`
Unrelated visitor used for preservation checks	`visitor@example.test` (seeded)
Configured admin	`admin@example.test`
Order A product	`productId: 1, quantity: 1`
Order B product	`productId: 2, quantity: 1`

Working notes

Write the actual values returned by Swagger. Example IDs shown later are illustrative only.

Value	Record here	Where it comes from
Visitor email	_____	Your unique sample email
Order A ID	_____	Order A response body `id`
Order A job ID	_____	`X-Notification-Job-ID` response header
Order B ID	_____	Order B response body `id`
Order B job ID	_____	`X-Notification-Job-ID` response header
Notification ID to modify/delete	_____	GET notifications response

Enum values used

Field	Number	Meaning
`channel`	4	InApp
`orderStatus`	10	Completed
`orderStatus`	11	Cancelled
`paymentStatus`	4	Paid
`fulfillmentStatus`	4	Packed
`deliveryStatus`	8	Delivered

4. Swagger execution convention

1. Expand the named endpoint and select **Try it out**.
2. Replace all example placeholders with values from your working notes.
3. Select **Execute**.
4. Record the response code, response body, and important response headers.

5. Compare the result with the expected output and pass criteria.

5. TC-B01 — Order A placement and asynchronous notifications

> **Business explanation:** Placing an order commits the order first, stores two notification job items in SQL, and publishes one small command to RabbitMQ. The worker uses the job ID to load the sensitive details from SQL and creates the final notification records.

5.1 Add a product to the visitor cart

1. In Swagger, execute `POST /api/v2/cart/items`.

```
{
  "visitorEmail": "swagger.smoke.202608071530@example.test",
  "productId": 1,
  "quantity": 1
}
```

Expected: HTTP 200. The response contains the same visitor email, at least one cart item, and a positive total.

```
{
  "visitorEmail": "swagger.smoke.202608071530@example.test",
  "items": [{ "id": 10, "productId": 1, "quantity": 1, "subtotal": 349.00 }],
  "totalAmount": 349.00
}
```

5.2 Place Order A

2. Execute `POST /api/v2/orders`.

```
{
  "visitorEmail": "swagger.smoke.202608071530@example.test"
}
```

Expected: HTTP 201. Save the response body `id` as Order A ID. In **Response headers**, save `X-Notification-Job-ID` and `X-Correlation-ID`. `X-Notification-Handoff` must be `Confirmed`.

```
HTTP 201 Created
X-Notification-Handoff: Confirmed
X-Notification-Job-ID: 11111111-2222-3333-4444-555555555555
X-Correlation-ID: 00-...

{
  "id": 101,
  "visitorEmail": "swagger.smoke.202608071530@example.test",
  "orderStatus": 2,
  "paymentStatus": 1,
  "fulfillmentStatus": 1,
  "deliveryStatus": 1,
  "totalAmount": 349.00,
  "items": [ ... ]
}
```

> **RabbitMQ expected:** The API has published one persistent command to exchange `nsa.notifications.commands.v1` using routing key `bulk.requested.v1`. With the worker running, the queue may return to zero too quickly to observe.

5.3 Poll the asynchronous job

3 Execute `GET /api/v2/notifications/bulk/{jobId}` with the saved Order A job ID. Repeat until the status is terminal.


```
{
  "jobId": "11111111-2222-3333-4444-555555555555",
  "status": "Completed",
  "totalCount": 2,
  "processedCount": 2,
  "succeededCount": 2,
  "failedCount": 0,
  "queuedAtUtc": "2026-08-07T07:30:00Z",
  "startedAtUtc": "2026-08-07T07:30:01Z",
  "completedAtUtc": "2026-08-07T07:30:01Z",
  "error": null,
  "correlationId": "00-..."
}
```


Expected: HTTP 200, `Completed`, total 2, processed 2, succeeded 2, failed 0.

5.4 Verify final notification records

4 Execute `GET /api/v2/notifications`. Set `orderId` to Order A ID; leave `recipientEmail` empty.


```
[
  {
    "id": 501,
    "recipientEmail": "admin@example.test",
    "channel": 4,
    "subject": "New order #101",
    "body": "Order #101 for swagger.smoke.202608071530@example.test. Status: Pending; ...",
    "orderId": 101,
    "isRead": false,
    "createdAtUtc": "2026-08-07T07:30:01Z",
    "sentAtUtc": null
  },
  {
    "id": 502,
    "recipientEmail": "swagger.smoke.202608071530@example.test",
    "channel": 4,
    "subject": "Order #101 received",
    "body": "Order #101 for swagger.smoke.202608071530@example.test. Status: Pending; ...",
    "orderId": 101,
    "isRead": false,
    "createdAtUtc": "2026-08-07T07:30:01Z",
    "sentAtUtc": null
  }
]
```


PASS when: exactly two records exist for Order A—one admin “New order” notification and one visitor “received” notification; both are InApp (`channel: 4`). Job correlation and counts are correct.

FAIL clues: `Queued` that never advances usually means the worker is unavailable. `PublishFailed` or an unconfirmed handoff points to RabbitMQ publication. `DeadLettered` means repeated worker processing failure.

6. TC-B02 — Admin changes Order A details

Business explanation: The admin submits all four tracking dimensions together. The API saves the order and creates one new InApp notification for the visitor and one for the admin. This path is synchronous and does not use RabbitMQ.

1. Execute `PATCH /api/v2/orders/{id}/status` using Order A ID.

```
{
  "orderStatus": 10,
  "paymentStatus": 4,
  "fulfillmentStatus": 4,
  "deliveryStatus": 8
}
```

Expected: HTTP 200 with the four supplied numeric values.

```
{
  "id": 101,
  "visitorEmail": "swagger.smoke.202608071530@example.test",
  "orderStatus": 10,
  "paymentStatus": 4,
  "fulfillmentStatus": 4,
  "deliveryStatus": 8,
  "totalAmount": 349.00,
  "items": [ ... ]
}
```

2. Execute `GET /api/v2/orders/{id}`. Confirm the same four values remain persisted.
3. Execute `GET /api/v2/notifications` with Order A ID.

Expected: four records total: two initial notifications plus two records whose subject is `Order #101 status updated`. One update recipient is `admin@example.test`; the other is the smoke visitor. Both bodies contain:

```
Status: Completed
Payment: Paid
Fulfillment: Packed
Delivery: Delivered
```

RabbitMQ expected: No new command is published. Main queue and DLQ totals should not change because status-update notifications are written synchronously by the API.

PASS when: all four order fields persist and exactly two new status-update notifications exist with correct recipients and content.

7. TC-B03 — Visitor cancellation on Order B

> **Business explanation:** A visitor-specific endpoint prevents the visitor from controlling payment, fulfillment, or delivery. It verifies the supplied email owns the order, changes only the overall order status, and notifies the admin once.

7.1 Create an independent Order B

Repeat the Order A placement steps with product 2:

1. `POST /api/v2/cart/items`

```
{
  "visitorEmail": "swagger.smoke.202608071530@example.test",
  "productId": 2,
  "quantity": 1
}
```

2. `POST /api/v2/orders` with the same visitor email. Save Order B ID and job ID.
3. Poll `GET /api/v2/notifications/bulk/{jobId}` until `Completed`. Verify two initial Order B notifications.

7.2 Cancel as the owner

4. Execute `PATCH /api/v2/orders/{id}/cancel` with Order B ID.

```
{
  "visitorEmail": "swagger.smoke.202608071530@example.test"
}
```

Expected: HTTP 200. `orderStatus` becomes 11 (Cancelled); initial payment, fulfillment, and delivery values remain unchanged.

```
{
  "id": 102,
  "visitorEmail": "swagger.smoke.202608071530@example.test",
  "orderStatus": 11,
  "paymentStatus": 1,
  "fulfillmentStatus": 1,
  "deliveryStatus": 1,
  "totalAmount": 429.00,
  "items": [ ... ]
}
```

5. GET notifications filtered by Order B ID. Expect three records: two initial placement records plus:

```
{
  "id": 505,
  "recipientEmail": "admin@example.test",
  "channel": 4,
  "subject": "Order #102 cancelled by visitor",
  "body": "Order #102 for swagger.smoke.202608071530@example.test was cancelled by the visitor. ... Status:
Cancelled; ...",
  "orderId": 102,
  "isRead": false,
  "sentAtUtc": null
}
```

6. Execute the same cancellation request again, then GET Order B notifications again.

> **PASS when:** the repeat response is HTTP 200 and there is still exactly one `cancelled by visitor` notification. No duplicate was created.

RabbitMQ expected: Cancellation itself publishes nothing. Only Order B placement used the broker. Queue totals should not change when the cancellation endpoint runs.

8. TC-B04 — Get notifications

> **Business explanation:** Notifications must be observable by recipient, order relationship, and stable record ID. These reads prove the records are persisted rather than present only in logs or broker memory.

8.1 List and filter

Execute `GET /api/v2/notifications` with these combinations:

recipientEmail	orderId	Expected
Smoke visitor	empty	Visitor-addressed notifications across Order A and B.
empty	Order A ID	Four Order A records after the admin update.
`admin@example.test`	Order B ID	New-order and cancellation notifications for Order B.
Smoke visitor	Order B ID	Visitor's Order B "received" record only.

When both filters are supplied, both conditions must match.

8.2 Get by ID

From the Order B response, copy the ID of the admin cancellation notification. Execute `GET /api/v2/notifications/{id}`.


```
{
  "id": 505,
  "recipientEmail": "admin@example.test",
  "channel": 4,
  "subject": "Order #102 cancelled by visitor",
  "body": "Order #102 for swagger.smoke.202608071530@example.test was cancelled by the visitor. ...",
  "orderId": 102,
  "isRead": false,
  "createdAtUtc": "2026-08-07T07:34:00Z",
  "sentAtUtc": null
}
```


> **PASS when:** HTTP 200 and every returned field matches the selected list item. A nonexistent ID returns HTTP 404 with `application/problem+json`.

> **RabbitMQ expected:** Read endpoints query SQL only. Broker queue counts and message rates remain unchanged.

9. TC-B05 — Modify a notification

Business explanation: This verifies the actual notification update endpoint. It uses full replacement semantics; even when only the read state changes, all editable values must be supplied.

1. Use the cancellation notification returned by GET. Copy its actual recipient, channel, subject, body, and order ID.
2. Execute `PUT /api/v2/notifications/{id}` using its ID. Set `isRead` to true.

```
{
  "recipientEmail": "admin@example.test",
  "channel": 4,
  "subject": "Order #102 cancelled by visitor",
  "body": "Order #102 for swagger.smoke.202608071530@example.test was cancelled by the visitor. Order #102 for
swagger.smoke.202608071530@example.test. Status: Cancelled; Payment: Unpaid; Fulfillment: NotStarted; Delivery:
WaitingForRider. Total: $429.00. Items: Butter Croissant Box x 1 @ $429.00 = $429.00",
  "orderId": 102,
  "isRead": true
}
```

Replace sample values: Use the actual notification body and IDs returned in your environment. Currency formatting and generated IDs may differ. Do not omit fields; this is a full `PUT`, not a partial patch.

Expected: HTTP 200. The same notification ID is returned, `isRead` is true, and all other supplied values remain intact.

3. GET the notification by ID again and verify `isRead: true`.

PASS when: update and subsequent GET agree, and no second notification record is created.

RabbitMQ expected: No broker message; the API updates the SQL record directly.

10. TC-B06 — Individual and visitor-wide deletion

10.1 Delete one notification

> **Business explanation:** Individual deletion removes one selected notification without deleting its order or sibling notifications.

1. Execute `DELETE /api/v2/notifications/{id}` with the modified notification ID.

Expected: HTTP 204 with no response body.

2. GET the same ID.

```
HTTP 404 Not Found
Content-Type: application/problem+json

{
  "status": 404,
  "title": "The requested operation could not be completed.",
  "instance": "/api/v2/notifications/505",
  "traceId": "..."
}
```

3. GET Order B by ID. It must still return HTTP 200.

> **PASS when:** selected notification is gone, Order B remains, and other Order B notifications remain.

10.2 Delete every notification related to the smoke visitor

> **Business explanation:** This administrative operation deletes records addressed directly to the visitor and admin records related through that visitor's orders. It does not delete orders or bulk-job history.

4. Execute `DELETE /api/v2/notifications/visitor`. Set query parameter `visitorEmail` to the unique smoke visitor.

Expected: HTTP 204. Repeating the same DELETE also returns 204.

5. GET notifications for Order A and Order B. Both results must be empty arrays.

```
[]
```

6. GET Order A and B by ID. Both must still return HTTP 200.

7. GET notifications with `recipientEmail=visitor@example.test`. Seeded unrelated records must still exist.

> **PASS when:** smoke-visitor-related notifications are gone; both smoke orders and unrelated visitor records remain.

> **RabbitMQ expected:** Neither delete operation publishes a broker command. Existing completed job rows remain queryable until their retention cleanup; queue counts do not change.

11. RabbitMQ data flow

11.1 Happy-path sequence

```
1. Swagger POST /orders
       |
       ▼
2. API commits Order + BulkNotificationJob + 2 JobItems to SQL
       |
       ▼
3. API publishes one persistent command with publisher confirmation
       |  exchange: nsa.notifications.commands.v1 (direct, durable)
       |  routing:  bulk.requested.v1
       ▼
4. queue: nsa.notifications.bulk.v1 (durable quorum)
       |
       ▼  worker consumes; autoAck=false; prefetch=1
5. Worker loads recipient/subject/body from SQL using JobId
       |
       ▼
6. Worker creates notification records and saves job progress to SQL
       |
       ▼
7. Worker ACKs only after successful persistence; queue removes command
```

> **Privacy boundary:** The broker command intentionally contains no recipient address, subject, or body. Those details remain in SQL Server.

11.2 Command body and AMQP metadata

```
{
  "schemaVersion": 1,
  "messageId": "aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee",
  "jobId": "11111111-2222-3333-4444-555555555555",
  "correlationId": "00-...",
  "createdAtUtc": "2026-08-07T07:30:00Z"
}

AMQP type: nsa.notifications.bulk-requested.v1
delivery mode: persistent
mandatory routing: enabled
publisher confirmations: enabled
```

11.3 Topology

Element	Name / setting
Command exchange	`nsa.notifications.commands.v1`, direct, durable
Main queue	`nsa.notifications.bulk.v1`, durable quorum
Command routing key	`bulk.requested.v1`
Dead-letter exchange	`nsa.notifications.dead-letter`, direct, durable
Dead-letter queue	`nsa.notifications.bulk.dlq`, durable quorum
DLQ routing key	`bulk.dead-letter.v1`
Application attempts	3 total; retry header `x-retry-count`

12. TC-R01 — Observe queued delivery and ACK

RabbitMQ test explanation: With a fast worker, the queue may appear empty even though messaging worked. Temporarily stopping only the worker lets you prove that the API publishes durable work and that the worker later consumes and acknowledges it.

Preparation

1. Open RabbitMQ Management: `http://localhost:15672`.
2. Open **Queues and Streams** → `nsa.notifications.bulk.v1`.
3. Record the current **Ready**, **Unacked**, and **Total** counts.
4. Stop only the worker:

```
docker compose --env-file notification-api/.env `
  -f notification-api/compose.week3.yml stop worker
```

Publish through Swagger

1. Use a new unique visitor email.
2. Add a cart item through Swagger.
3. Place the order through Swagger and verify HTTP 201 plus `X-Notification-Handoff: Confirmed`.
4. GET the job status: it should remain `Queued` while the worker is stopped.
5. Refresh the main queue page.

Expected while stopped: **Ready** increases by 1, **Unacked** is 0, and the job is Queued. No final notification records exist for the new order yet.

Resume processing

```
docker compose --env-file notification-api/.env `
  -f notification-api/compose.week3.yml start worker
```

1. Refresh the queue page; **Unacked** may briefly show 1.
2. After processing, Ready and Unacked return to their baseline.
3. In Swagger, poll job status until Completed and GET the two notification records.

PASS when: the command is Ready while the worker is stopped, disappears only after the worker returns, the job completes 2/2, and both notification records exist.

Do not purge the queue: Purging would destroy queued test work and invalidate the delivery observation.

13. TC-R02 — Retry and dead-letter flow

> **RabbitMQ test explanation:** A poison command must not retry forever. The worker republishes a retry only after a failure, acknowledges the original only after retry publication succeeds, and rejects the third failed attempt so RabbitMQ routes it to the DLQ.

> **Isolated local test only:** Failure injection is disabled by default and must never be enabled in a shared or production-like environment.

13.1 Enable the documented poison subject

```
$env:WEEK3_FAILURE_INJECTION_SUBJECT = '[week3-poison]'
docker compose --env-file notification-api/.env `
  -f notification-api/compose.week3.yml up --detach --force-recreate worker
```

13.2 Submit the poison job through Swagger

Execute `POST /api/v2/notifications/bulk`:

```
{
  "notifications": [
    {
      "recipientEmail": "poison.swagger@example.test",
      "channel": 4,
      "subject": "[week3-poison]",
      "body": "Intentional local smoke test of retry and DLQ routing.",
      "orderId": null
    }
  ]
}
```

Expected Swagger response: HTTP 202 with a job ID, `status: "Queued"`, total 1, processed 0.

13.3 Observe retry and terminal output

Poll `GET /api/v2/notifications/bulk/{jobId}`. The state may pass through `Processing` and `Retrying`. Final expected response:

```
{
  "jobId": "...",
  "status": "DeadLettered",
  "totalCount": 1,
  "processedCount": 0,
  "succeededCount": 0,
  "failedCount": 0,
  "completedAtUtc": "2026-08-07T07:45:00Z",
  "error": "Processing failed after 3 attempts; the command was routed to the dead-letter queue.",
  "correlationId": "..."
}
```

In RabbitMQ Management:

- Main queue returns to its baseline.
- `nsa.notifications.bulk.dlq` Ready count increases by 1.
- The final command has `x-retry-count: 2`; RabbitMQ adds dead-letter metadata such as `x-death`.
- No notification record is created for `poison.swagger@example.test`.

> **PASS when:** exactly three application attempts occur, SQL status becomes DeadLettered, the command reaches the named DLQ, and no final notification is persisted.

13.4 Disable failure injection

```
Remove-Item Env:WEEK3_FAILURE_INJECTION_SUBJECT -ErrorAction SilentlyContinue
docker compose --env-file notification-api/.env `
  -f notification-api/compose.week3.yml up --detach --force-recreate worker
```

Confirm the worker is healthy before continuing. The DLQ message remains evidence until intentionally inspected/replayed/removed.

> **Safe DLQ inspection:** In RabbitMQ Management, use **Get messages** with a requeue option when available. Do not acknowledge/remove the message unless cleanup is intentional.

14. RabbitMQ attempt model

Delivery	Header	On processing failure
Attempt 1	absent or `0`	Republish with `x-retry-count: 1`; ACK original after publish succeeds.
Attempt 2	`1`	Republish with `x-retry-count: 2`; ACK original after publish succeeds.
Attempt 3	`2`	Persist DeadLettered and reject without requeue; DLX routes to DLQ.

If retry publication itself fails, the worker returns the original delivery to RabbitMQ instead of acknowledging it. A malformed, unsupported-schema, wrong-type, or missing-identifier command is rejected directly to the DLQ without application retry.

15. Evidence and pass/fail record

Test	Result	Evidence to retain
TC-B01 Order placement	PASS / FAIL	Order ID, job ID, headers, completed job, 2 notification IDs.
TC-B02 Admin details	PASS / FAIL	Updated order response and 2 update notification IDs.
TC-B03 Cancellation	PASS / FAIL	Cancelled order, preserved statuses, single admin notification.
TC-B04 Get	PASS / FAIL	Filter and get-by-ID output.
TC-B05 Modify	PASS / FAIL	PUT and subsequent GET output.
TC-B06 Delete	PASS / FAIL	204/404, empty related lists, surviving orders/unrelated data.
TC-R01 Queue/ACK	PASS / FAIL	Queue counts before/stopped/after, job transition.
TC-R02 Retry/DLQ	PASS / FAIL	Job ID, correlation ID, worker logs, DLQ count/header.

16. Troubleshooting

```
docker compose --env-file notification-api/.env `
  -f notification-api/compose.week3.yml ps

docker compose --env-file notification-api/.env `
  -f notification-api/compose.week3.yml logs --tail 200 api worker rabbitmq sqlserver
```

Observed output	Meaning	Action
Swagger cannot load	API is unavailable or port mapping failed.	Check container state, API logs, and port 8080.
POST order returns 400	Cart empty or request invalid.	GET the cart using the exact same visitor email; add a valid seeded product.
Handoff Unconfirmed	Order committed, but broker confirmation was ambiguous/failed.	Check RabbitMQ readiness, credentials, exchange/queue, and API publisher logs.
Job remains Queued	No active worker consumption.	Start worker; check main queue consumer count and worker SQL connectivity.
Job Completed but no records	Unexpected persistence/query mismatch.	Verify order ID filter and inspect worker/API logs with correlation ID.
Unexpected notification count	Visitor email or order reused.	Use a new unique email and new orders; compare subjects and order IDs.
Cancellation returns 404	Order missing or supplied email does not own it.	Copy visitor email exactly from GET order output.
PUT returns 400	Full replacement omitted/invalid a required field.	Copy all editable values from GET, then change only `isRead`.
DLQ does not increase	Failure injection absent, wrong subject, or worker not recreated.	Check worker environment/logs and exact case-sensitive poison subject.

17. Seven recommended additional smoke tests

1. **Cancellation ownership:** supply another visitor's email; expect 404, unchanged order, no notification.
2. **Broker outage on order placement:** stop RabbitMQ; verify committed order behavior and unconfirmed/rejected handoff reporting.
3. **API restart after broker acceptance:** inspect ambiguous publisher-confirm behavior and retained job observability.
4. **Malformed request validation:** invalid email and undefined enum values must return 400 Problem Details.
5. **Malformed broker command:** in an isolated environment, publish invalid JSON and confirm direct DLQ routing without retries.
6. **Dependency readiness:** stop SQL or RabbitMQ separately; liveness remains process-focused while readiness becomes unhealthy.
7. **Unrelated-data preservation:** create a second visitor's notification, delete the smoke visitor's history, and confirm the second record remains.

18. Cleanup

The visitor-wide delete removes smoke notifications but intentionally preserves orders and job history. Use unique emails so retained local smoke data is easy to identify. Stop the stack without deleting volumes:

```
docker compose --env-file notification-api/.env `
  -f notification-api/compose.week3.yml down
```

> **Data safety:** Do not add `--volumes` unless destroying all local SQL Server and RabbitMQ data is explicitly intended.
